Dan Glasses Master Product + Engineering Handoff (Canonical v1)

> *Project: Dan Glasses by DanLab*
> *Purpose: Single source of truth for product, deployment, architecture, engineering, and acceptance*
> *Last updated: 2026-04-16*
> *Status: Canonical handoff doc for developer agents and human implementers*

Table of Contents

1. Executive Summary

Dan Glasses v1 is a developer-first, offline-capable product delivered as a **single Linux app experience** backed by a native Linux installation. The visible product is a **Tauri supervisor app** called `dan-glasses-app`. Under the hood, the `.deb` installer sets up a managed local runtime consisting of OpenClaw plus dedicated local services for perception, audio, memory, and guarded OS execution.

This document replaces older speculative guidance and locks the following decisions:

- Product shell: `Tauri` desktop supervisor app
- Install format: signed `.deb` package with `systemd`
- Primary targets:
  - Redax `aarch64` device running Debian/Ubuntu
  - standard Linux laptop `x86_64` for developer mode
- Runtime model: `OpenClaw` is the primary orchestration runtime
- Camera support: generic `V4L2` provider with adapters for USB/UVC, integrated laptop webcams, and CSI-backed devices
- Voice stack: `whisper.cpp` for STT and `KittenTTS` for TTS
- Vision stack: `LFM2.5-VL-450M` primary, Gemma fallback
- Memory source of truth: `SQLite + Markdown mirror + local vector index`
- Model delivery: app installs first, models download on first run
- Update strategy: APT repository and signed package upgrades
- Install permissions: `sudo` is allowed and expected for full installation

2. Product Requirements Document

2.1 Product summary

Dan Glasses is a local-first OS copilot for developers. It observes the current device context through camera and audio, stores relevant context locally, and lets the user ask for help or trigger actions through OpenClaw with strong guardrails.

2.2 Primary users

1. Developer on a normal Linux laptop
   Uses a built-in webcam or USB webcam and wants coding help, screen understanding, local memory, and guarded system automation.

2. Developer on Redax-based Dan Glasses hardware
   Uses a board-connected or bridged camera plus local compute to run the same stack in a more wearable form factor.

3. Internal builder/operator
   Needs a packageable, debuggable, supportable runtime that can be installed, upgraded, and recovered using normal Linux tooling.

2.3 Core user problems

- Installing a local multimodal dev assistant today is fragmented and fragile.
- Camera, audio, models, memory, and tool execution are usually separate manual setup steps.
- Existing local assistants rarely provide good OS-level workflows with strong auditability.
- Memory systems are either too opaque, too cloud-dependent, or too hard to inspect manually.

2.4 Product goals

- One install flow that produces a working local system.
- One visible Linux app that lets the user operate the full stack.
- Offline-capable developer workflows after initial setup.
- Strong local memory with human-readable inspection.
- Broad Linux tooling access with clear policy and audit.

2.5 Day-one user journeys

Journey A: Laptop install and first run

1. User downloads and installs `dan-glasses_<version>_<arch>.deb`.
2. Installer creates services, data directories, and the desktop app entry.
3. User launches `dan-glasses-app`.
4. App detects hardware, cameras, and audio devices.
5. App downloads required models and verifies checksums.
6. App starts services and shows health.
7. User sees camera preview, microphone status, model readiness, and memory status.
8. User starts a developer session with OpenClaw.

Journey B: Developer assistance

1. Camera observes terminal/editor/screen.
2. Perception pipeline emits structured observations.
3. OpenClaw chooses whether to remember, notify, or act.
4. User speaks or types a request.
5. OpenClaw uses memory and guarded OS tools.
6. Result is shown in UI and optionally spoken aloud.

Journey C: Recovery and upgrade

1. User receives a new signed package through the APT repository.
2. Package upgrade preserves config, memory, and downloaded models.
3. Installer scripts re-run safely.
4. Services restart cleanly and app reports current version and health.

2.6 Success metrics

- Fresh install to healthy first run: under 10 minutes on broadband
- First-run model bootstrap completes without manual shell commands
- Offline startup works after models are installed
- Standard developer request succeeds without leaving the app
- Camera enumeration succeeds across laptop webcam, USB camera, and CSI-backed device classes
- Package upgrade preserves state with no manual repair

2.7 User flow diagram

flowchart TD A[User installs signed .deb] --> B[Launch dan-glasses-app] B --> C[Bootstrap wizard starts] C --> D[Detect OS, arch, cameras, audio] D --> E[Fetch and verify model manifest] E --> F[Download required local models] F --> G[Initialize memory stores] G --> H[Start local services] H --> I[Health checks pass] I --> J[User starts developer session] J --> K[Camera and audio feed local context] K --> L[OpenClaw decides remember / notify / act] L --> M[User asks by voice or text] M --> N[OpenClaw uses tools and memory] N --> O[Answer shown in UI and optionally spoken] O --> P[State persists locally]

3. MVPD: Minimum Viable Product Definition

3.1 v1 must do

- Install via native `.deb`
- Create and manage local system services
- Run OpenClaw locally
- Detect at least one compatible camera through V4L2
- Detect at least one microphone and output device
- Download required models on first run
- Support offline STT, TTS, perception, memory, and OS tooling after bootstrap
- Provide a Tauri UI for status, controls, logs, and configuration
- Store memory in SQLite and Markdown locally
- Support guarded file, shell, git, process, network, and memory actions

3.2 v1 must not depend on

- Cloud inference
- Cloud memory as source of truth
- Manual model placement by the user
- Manual editing of `systemd` units or Linux permission files after install
- Per-camera custom code paths as the primary camera strategy

3.3 v1 exact user-visible behaviors

Install flow

- User installs the package with `sudo apt install ./dan-glasses_<version>.deb`
- Installer creates a `dan-glasses` service account/group if needed
- Installer creates required directories
- Installer installs `systemd` units and desktop launcher
- Installer does not download large models yet

First-run onboarding

- App opens to a bootstrap wizard
- Wizard probes architecture, camera devices, audio devices, storage budget, and network
- Wizard selects a default hardware profile
- Wizard downloads only required model set for that profile
- Wizard verifies hashes and writes a model manifest
- Wizard starts services and validates health

Camera/audio/model auto-setup

- Camera auto-detection happens via V4L2 enumeration

- Audio auto-detection happens via local audio device enumeration
- Model bootstrap uses a signed manifest and checksum verification
- User can override defaults from the app

Developer workflows

- File inspection and edits
- `git status`, `git diff`, branch-safe workflows
- Process listing and restart
- Local network diagnostics
- Memory lookup and note creation
- Screen/camera context understanding

Offline behavior

- If models are already installed, all core flows continue offline
- Optional update checks and remote model fetch are skipped when offline
- Memory writes and local retrieval continue offline

4. Target Platform and Deployment Decisions

4.1 Primary OS baseline

Debian/Ubuntu on Redax is the primary deployment baseline for v1 because it supports:

- native `.deb` installation
- `systemd` services
- normal camera/audio device integration
- standard packaging and upgrade tools
- faster developer iteration than a custom image-first approach

4.2 Why not Buildroot/Yocto first

Buildroot/Yocto remain valid later for a dedicated device image, but they are not the primary v1 deployment baseline because they increase cost for:

- package upgrades
- service lifecycle management
- developer tooling support
- supportability on normal laptops

4.3 Why not Flatpak/AppImage first

They are not primary targets for v1 because this product needs:

- privileged installer actions
- `systemd` unit creation
- device rule setup
- persistent host-level service management

4.4 Supported architectures

- `x86_64-unknown-linux-gnu`
- `aarch64-unknown-linux-gnu`

These are separate packaged builds with the same service topology and UI semantics.

4.5 Non-negotiable build principles

- One user-facing Linux app, many managed local services.
- No cloud dependency for core functionality after bootstrap.
- OpenClaw remains the only orchestration runtime for v1.
- Memory is local-first and inspectable by default.

- Camera handling is generic through V4L2-first adapters.
- Install and upgrade flows must be operationally boring and supportable.
- Dangerous OS actions must be policy-controlled and auditable.

4.6 Redax reference hardware profile

Redax is still treated as a working board name, not a publicly finalized hardware spec. For implementation planning, use this reference profile:

Component	Reference target	Why it matters
CPU arch	`aarch64`	aligns with Redax package target
RAM	8 GB preferred, 4 GB minimum for reduced profile	enough for local models and services
Storage	128 GB eMMC or SSD preferred	model storage + memory + logs
Camera	CSI-backed sensor or UVC camera	covered by generic V4L2 layer
Audio	one working input and output device	minimum STT/TTS path
OS	Debian/Ubuntu family	matches package and service strategy
Service manager	`systemd`	required for v1 operational model

4.7 Development mission summary

If a developer or agent reads only one paragraph before starting work, it should be this:

Dan Glasses v1 is a packaged local multimodal developer copilot. The system installs as a signed Debian package, starts a Tauri supervisor app, provisions a set of local services, auto-detects camera/audio hardware, downloads and verifies local models on first run, then uses OpenClaw plus perception, memory, voice, and guarded OS tools to deliver a fully local developer assistance workflow on both Redax hardware and normal Linux laptops.

5. Single Linux App Architecture

5.1 Product shape

The product is a single app from the user perspective, but a multi-service runtime from the system perspective.

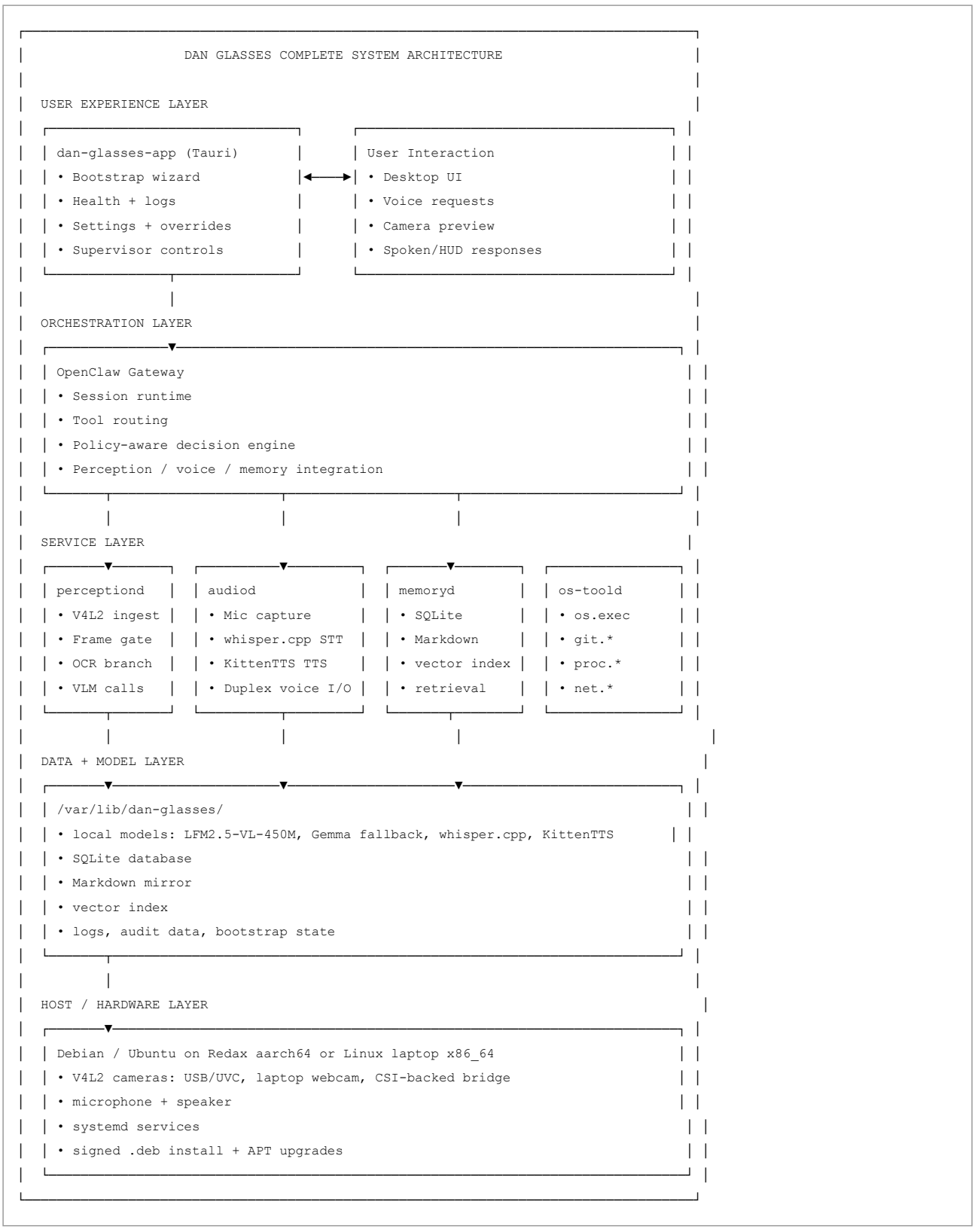
User-facing component

- `dan-glasses-app`
  - Tauri desktop application
  - launcher, supervisor UI, logs, health, settings, install/bootstrap wizard

Managed runtime components

- `openclaw-gateway`
  - orchestration runtime, sessions, tool routing
- `perceptiond`
  - camera ingest, frame gating, OCR, VLM inference dispatch
- `audiod`
  - microphone capture, `whisper.cpp`, `KittenTTS`
- `memoryd`
  - SQLite, Markdown mirror, vector writes and queries
- `os-toold`
  - guarded system tool executor
- `dan-glassesctl` (optional but recommended)
  - privileged helper for service control and host-level actions

5.2 Complete system architecture



5.2A Expanded layered architecture

5.3 Process topology

flowchart LR UI[dan-glasses-app\nTauri supervisor UI] CTL[dan-glassesctl\noptional privileged helper] OC[openclaw-gateway] P[perceptiond] A[audiod] M[memoryd] O[os-toold] CAM[V4L2 camera devices] MIC[Audio input] OUT[Speaker / HUD] DB[(SQLite)] MD[Markdown mirror] VEC[(Vector index)] UI --> CTL UI --> OC UI --> P UI --> A UI --> M UI --> O CAM --> P MIC --> A A --> OUT P --> OC A --> OC OC --> O OC --> M M --> DB M --> MD M --> VEC OC --> OUT

5.4 End-to-end runtime workflow

flowchart LR CAM[V4L2 Camera] --> P[perceptiond] MIC[Microphone] --> A[audiod] P --> OCR[OCR branch] P --> VLM[LFM2.5-VL-450M] VLM --> OC[OpenClaw] OCR --> OC A --> STT[whisper.cpp] STT --> OC OC --> MEM[memoryd] OC --> TOOL[os-toold] MEM --> DB[(SQLite + Markdown + Vectors)] TOOL --> HOST[Linux host actions] OC --> TTS[KittenTTS] TTS --> OUT[Speaker / HUD / UI]

5.5 Filesystem layout

- `/opt/dan-glasses/`
  - app bundle
  - sidecar binaries
  - static manifests
- `/var/lib/dan-glasses/`
  - models
  - SQLite database
  - vector index
  - logs
  - bootstrap state
- `/etc/dan-glasses/`
  - config files
  - policy files

    - model manifests
- `/usr/lib/systemd/system/`
    - service units
- `/usr/share/applications/`
    - desktop launcher

5.6 Ownership and permissions

- service account: `dan-glasses`
- service group: `dan-glasses`
- app UI runs in user session
- services run under least-privilege users where practical
- model/data directories owned by `dan-glasses`
- Markdown export location configurable but defaulted under `/var/lib/dan-glasses/markdown/`

5.7 Exact systemd units

The package should install these units:

- `dan-glasses-openclaw.service`
- `dan-glasses-perception.service`
- `dan-glasses-audiod.service`
- `dan-glasses-memoryd.service`
- `dan-glasses-os-toold.service`
- `dan-glasses-bootstrap.service` (one-shot or manually triggered helper)

Unit expectations:

- `openclaw`, `memoryd`, and `os-toold` should come up before the UI is considered healthy
- `perceptiond` and `audiod` may enter degraded mode without marking the whole install failed
- unit logs must be visible through both `journalctl` and the Tauri app log viewer

5.8 Recommended implementation stack per component

To reduce ambiguity for the developer agent, the recommended v1 implementation stack is:

- `dan-glasses-app`
    - Tauri v2
    - TypeScript + React frontend
    - Rust host layer for native integration
- `openclaw-gateway`
    - Node.js 24
- `perceptiond`
    - Python 3.11
    - OpenCV + GStreamer integration
- `audiod`
    - Python 3.11
    - thin wrappers around `whisper.cpp` and `KittenTTS`
- `memoryd`
    - Python 3.11
    - SQLite + Markdown writer + vector index integration
- `os-toold`
    - Rust preferred for stricter command-policy enforcement
- `dan-glassesctl`
    - Rust or POSIX shell wrapper with minimal surface area

5.9 Complete developer assistance workflow

flowchart TD A[Camera or user voice input] --> B[perceptiond / audiod] B --> C[Structured observation or transcript] C --> D[OpenClaw session] D --> E{Need memory?} E -- Yes --> F[memoryd lookup] E -- No --> G[Continue] F --> G G --> H{Need tool execution?} H -- Yes --> I[os-toold or other tool] H -- No --> J[Compose response] I --> J J --> K[memoryd append if needed] K --> L[UI response + optional TTS]

6. Install, Bootstrap, and Auto-Setup Pipeline

6.1 Package install responsibilities

The `.deb` `postinst` script must:

1. create the `dan-glasses` user/group if absent
2. create required directories
3. set ownership and permissions
4. install `systemd` units
5. install the desktop launcher
6. install or update default config files
7. install udev rules if required for device handling
8. reload `systemd`
9. enable services that should persist across reboot
10. leave model download for first run

6.2 Installer script requirements

- idempotent
- safe on upgrade
- safe on reinstall
- safe on purge/reinstall
- must not delete user memory/config during upgrade
- must emit readable logs on failure

6.3 First-run bootstrap sequence

The first run of `dan-glasses-app` performs the following in order:

1. detect host architecture (`x86_64` or `aarch64`)
2. detect OS/version and confirm supported baseline
3. enumerate cameras via V4L2
4. enumerate microphones and speakers
5. measure available disk space
6. choose bootstrap hardware profile
7. resolve required model manifest
8. download models with checksum validation
9. initialize SQLite schema
10. initialize Markdown mirror directory structure
11. initialize vector index
12. start or restart services
13. run service health checks
14. present a final readiness screen

6.4 Bootstrap failure handling

- If no supported camera is found:
  - app enters `camera_degraded` mode
  - voice and memory features remain available
- If no microphone is found:
  - app enters `audio_degraded` mode
  - typed interaction still works
- If model download fails:
  - app stores resume state
  - app retries on next run or on explicit retry
- If disk is insufficient:
  - app offers a reduced profile with fewer models

6.5 Bootstrap health checks

Each service must expose a local health check that includes:

- service name
- version
- running status
- last heartbeat
- degraded reason if any

6.6 Bootstrap artifacts written to disk

The bootstrap flow must produce these durable artifacts:

- `/etc/dan-glasses/app.toml`
- `/etc/dan-glasses/policy.toml`
- `/etc/dan-glasses/models.json`
- `/var/lib/dan-glasses/bootstrap/state.json`
- `/var/lib/dan-glasses/bootstrap/downloads/`
- `/var/lib/dan-glasses/sqlite/dan_glasses.db`
- `/var/lib/dan-glasses/markdown/`
- `/var/lib/dan-glasses/vectors/`

The app should treat bootstrap as complete only after all of these exist and service health checks pass.

6.7 Install and bootstrap flow diagram

flowchart TD A[apt install dan-glasses.deb] --> B[postinst runs] B --> C[Create user/group] C --> D[Create dirs and config] D --> E[Install systemd units and desktop entry] E --> F[User launches dan-glasses-app] F --> G[Bootstrap probe] G --> H[Detect arch, OS, cameras, audio] H --> I[Select bootstrap profile] I --> J[Download and verify model files] J --> K[Initialize SQLite, Markdown, vectors] K --> L[Start or restart services] L --> M[Run health checks] M --> N{Healthy?} N -- Yes --> O[Ready for developer session] N -- No --> P[Enter degraded mode and surface remediation]

7. Runtime Service Contracts

7.1 `CameraDeviceDescriptor`

```
type CameraDeviceDescriptor = {
  id: string;
  path: string;
  kind: "usb_uvc" | "laptop_webcam" | "csi_bridge" | "unknown";
  bus: "usb" | "platform" | "pci" | "i2c" | "unknown";
  capabilities: {
    formats: string[];
    resolutions: Array<{ width: number; height: number; fps: number[] }>;
  };
  default_priority: number;
};
```

7.2 Camera provider API

`perceptiond` must implement:

- `enumerateDevices(): CameraDeviceDescriptor[]`
- `probeDevice(id): ProbeResult`
- `selectDefault(devices): string`
- `openStream(deviceId, profile): StreamHandle`
- `captureFrame(deviceId): FrameRef`

Classification rules:

- `usb_uvc`: external USB cameras exposed through UVC/V4L2
- `laptop_webcam`: integrated webcam heuristically identified via internal bus/sysfs topology
- `csi_bridge`: CSI-backed or bridge-backed sensor exposed through Linux media stack

7.3 `BootstrapPlan`

```
type BootstrapPlan = {
  arch: "x86_64" | "aarch64";
  camera_profile: "none" | "basic_webcam" | "multi_camera" | "csi_preferred";
  audio_profile: "none" | "basic_audio" | "beamforming_audio";
  model_manifest: string;
  memory_mode: "sqlite_markdown_vector";
};
```

7.4 Voice API

```
type SttRequest = {
  audio_chunk: string;
  language?: string;
};

type TtsRequest = {
  text: string;
  voice: string;
  rate?: number;
};
```

Required runtime calls:

- `stt.transcribe(audio_chunk)`
- `tts.speak(text, voice, rate)`

7.5 `PerceptionObservation`

```
type PerceptionObservation = {
  event_id: string;
  captured_at: string;
  source: "camera.webcam";
  dev_context: {
    likely_surface?: "terminal" | "editor" | "browser" | "filesystem" | "unknown";
    workspace_hint?: string;
    tool_hint?: string;
  };
  ocr_text?: string;
  action_hint?: "remember" | "notify" | "act" | "ignore";
  salience: number;
};
```

7.6 `MemoryWriteResult`

```
type MemoryWriteResult = {
  event_id: string;
  memory_id: string;
  vector_id: string;
  markdown_path: string;
  obsidian_mirror_status: "disabled" | "success" | "failed";
};
```

7.7 `CommandPolicyProfile`

```
type CommandPolicyProfile = "readonly" | "dev-standard" | "admin-guarded";
```

7.8 `ServiceHealth`

```
type ServiceHealth = {
  service: string;
  status: "starting" | "healthy" | "degraded" | "failed" | "stopped";
  version: string;
  last_heartbeat: string;
  degraded_reason?: string;
};
```

7.9 IPC contracts

Producer	Consumer	Transport	Contract
`perceptiond`	`openclaw-gateway`	WebSocket	`perception.observation` events
`audiod`	`openclaw-gateway`	WebSocket	`voice.transcript.partial`, `voice.transcript.final`
`openclaw-gateway`	`audiod`	local RPC	`voice.speak`
`openclaw-gateway`	`memoryd`	Unix socket or local RPC	read/write/search memory
`openclaw-gateway`	`os-toold`	Unix socket or local RPC	guarded command execution
`dan-glasses-app`	all services	local RPC + health endpoints	status/control/bootstrap
`dan-glasses-app`	`dan-glassesctl`	local command/helper	privileged install/control actions

7.10 Detailed schema examples

Example `CameraDeviceDescriptor`

```
{
  "id": "video0",
  "path": "/dev/video0",
  "kind": "usb_uvc",
  "bus": "usb",
  "capabilities": {
    "formats": ["MJPEG", "YUYV"],
    "resolutions": [
      { "width": 1280, "height": 720, "fps": [15, 30] },
      { "width": 1920, "height": 1080, "fps": [15, 30] }
    ]
  },
  "default_priority": 80
}
```

Example `PerceptionObservation`

```
{
  "event_id": "evt_2026_04_16_0001",
  "captured_at": "2026-04-16T12:00:00Z",
  "source": "camera.webcam",
  "dev_context": {
    "likely_surface": "terminal",
    "workspace_hint": "/home/dev/project",
    "tool_hint": "npm"
  },
  "ocr_text": "ERR! missing script: build",
  "action_hint": "notify",
  "salience": 0.86
}
```

Example `MemoryWriteResult`

```
{
  "event_id": "evt_2026_04_16_0001",
  "memory_id": "mem_2026_04_16_0001",
  "vector_id": "vec_2026_04_16_0001",
  "markdown_path": "/var/lib/dan-glasses/markdown/journal/2026-04-16/evt_2026_04_16_0001.md",
  "obsidian_mirror_status": "disabled"
}
```

Example `ServiceHealth`

```
{
  "service": "dan-glasses-perception.service",
  "status": "healthy",
  "version": "0.1.0",
  "last_heartbeat": "2026-04-16T12:05:00Z"
}
```

8. OpenClaw Developer Runtime and Policy Model

8.1 OpenClaw scope

OpenClaw is the only orchestration runtime in scope for v1. It is responsible for:

- user sessions
- tool routing
- decision logic
- integration with perception events
- integration with voice and memory

8.2 Tool surface

OpenClaw must expose:

- `os.exec`
- `os.read`
- `os.write`
- `git.status`
- `git.diff`
- `git.commit`
- `proc.list`
- `proc.restart`

- `proc.stop`
- `net.fetch`
- `net.check`
- `memory.lookup`
- `memory.append`
- `memory.summarize`
- `voice.transcribe`
- `voice.speak`
- `hud.show`
- `hud.clear`

8.3 Permission profiles

`readonly`

- read files
- read logs
- diagnostics
- search memory
- no mutating commands

`dev-standard`

- common edits
- package commands
- process restarts
- safe git operations
- mutating commands allowed under policy

`admin-guarded`

- privileged commands
- system-wide changes
- service overrides
- explicit user confirmation required

8.4 Guardrails

- denylist for destructive commands
- protected path rules for sensitive directories
- argument hashing and audit log
- per-command approval metadata
- emergency kill switch halting `os.exec`

8.5 Audit log

All mutating actions must record:

- timestamp
- initiating actor
- profile used
- tool name
- argument hash
- result code
- error summary if failed

9. Memory Architecture

9.1 Canonical source of truth

The v1 canonical memory architecture is:

- SQLite database for structured truth
- Markdown mirror for human readability and external interoperability
- local vector index for semantic retrieval

This replaces older guidance that treated Obsidian or advanced memory frameworks as the primary source of truth.

9.2 Memory write pipeline

Each accepted memory event writes:

1. append-only event row
2. normalized memory row
3. vector embedding row/index entry
4. Markdown mirror file
5. optional Obsidian mirror note if enabled

9.3 Minimum SQLite tables

- `events`
- `event_entities`
- `memories`
- `artifacts`
- `markdown_exports`
- `audit_log`
- `service_health`

9.4 Markdown mirror structure

```
/var/lib/dan-glasses/markdown/
  journal/
  memories/
  sessions/
  entities/
  summaries/
```

9.5 Adapter strategy

The following are adapters or experiments in v1, not the canonical core:

- Obsidian MCP
- Mem0
- MemPalace
- Hermes memory-provider integrations

They can be enabled only if they do not disrupt the canonical pipeline.

9.6 Memory retrieval contract

`memoryd` must support:

- lookup by semantic similarity
- lookup by recency
- lookup by entity or tag
- reconstruction of linked Markdown path from canonical memory IDs

9.7 Memory architecture diagram

flowchart LR OBS[PerceptionObservation] --> EVT[events table] OBS --> MEM[memories table] MEM --> MD[Markdown mirror] MEM --> VEC[Vector index] MEM --> ART[Optional artifact refs] MEM --> OBD[Optional Obsidian mirror] Q[Memory query] --> VEC Q --> EVT EVT --> PACK[Context pack] MEM --> PACK PACK -->

OC[OpenClaw]

9.8 Local storage layout and retention policy

Use this storage policy for v1:

Path	Purpose	Retention
`/var/lib/dan-glasses/sqlite/dan_glasses.db`	canonical structured memory	indefinite unless user resets
`/var/lib/dan-glasses/markdown/`	human-readable mirror	indefinite unless user resets
`/var/lib/dan-glasses/vectors/`	semantic retrieval index	rebuildable from canonical memory
`/var/lib/dan-glasses/logs/`	service and audit logs	rotate by size and age
`/var/lib/dan-glasses/artifacts/`	optional retained media	opt-in only
`/var/lib/dan-glasses/bootstrap/`	first-run state and downloads	kept for recovery/debug
`/run/dan-glasses/`	transient runtime cache, ring buffers	cleared on reboot

Retention rules:

- Raw media is off by default.
- Structured memory and Markdown mirror persist by default.
- Vector index may be rebuilt if corrupted.
- Artifact retention must be user-enabled and privacy-aware.
- Service logs should rotate automatically.

9.9 Memory write and read flow

flowchart LR OBS[Observation or user action] --> EVT[Append event row] EVT --> MEM[Write normalized memory row] MEM --> MD[Write Markdown mirror] MEM --> VEC[Write vector index entry] MEM --> OBD[Optional Obsidian export] Q[Lookup request] --> VEC Q --> EVT VEC --> PACK[Candidate context pack] EVT --> PACK PACK --> OC[OpenClaw reasoning]

10. Model, Camera, and Audio Runtime

10.1 Model policy

- primary VLM: `LFM2.5-VL-450M`
- fallback VLM: Gemma-compatible local model profile
- STT: `whisper.cpp`
- TTS: `KittenTTS`

10.2 Model delivery strategy

The `.deb` ships:

- app binaries
- sidecar binaries
- bootstrap manifests
- configs

The `.deb` does **not** ship large model weights by default.

10.3 First-run model manifest

A model manifest must define:

- model name
- version
- arch support
- required/optional flag
- size
- destination path
- checksum
- source URLs

Example shape:

```
{
  "models": [
    {
      "name": "lfm2.5-vl-450m",
      "required": true,
      "arches": ["x86_64", "aarch64"],
      "sha256": "..."
    }
  ]
}
```

10.4 Camera support policy

Camera support is **generic V4L2-first**, not camera-specific. v1 must support:

- laptop built-in webcam
- USB/UVC webcam
- CSI-backed device exposed through Linux media stack

10.5 Audio policy

The minimum shipping audio mode is:

- single working microphone
- single working output device
- push-to-talk or explicit recording trigger

Optional wake features are deferred.

10.6 Fallback behavior

- if LFM unavailable, use Gemma fallback
- if no camera, app remains operational in reduced mode
- if no audio, typed workflow remains operational
- if model storage low, app offers reduced profile

10.7 Perception and decision flow

flowchart TD A[Camera frame arrives] --> B[perceptiond frame gate] B --> C{Meaningful change?} C -- No --> D[Drop or buffer frame] C -- Yes --> E[ROI extraction] E --> F{Text heavy?} F -- Yes --> G[OCR branch] F -- No --> H[VLM branch] G --> I[Build structured observation] H --> I I --> J[Publish to OpenClaw] J --> K{Remember / Notify / Act / Ignore} K -- Remember --> L[memoryd write] K -- Notify --> M[UI / TTS response] K -- Act --> N[tool invocation] K -- Ignore --> O[No user interruption]

10.8 Voice roundtrip flow

flowchart TD A[User speaks] --> B[audiod captures audio] B --> C[whisper.cpp transcription] C --> D[OpenClaw interprets intent] D --> E{Need tools or memory?} E -- Yes --> F[memoryd / os-toold] E -- No --> G[Direct response] F --> H[OpenClaw response text] G --> H H --> I[KittenTTS synthesis] I --> J[Speaker output + UI response]

10.9 Performance targets and latency budget

These are target budgets for v1 planning, not guaranteed shipping benchmarks:

Stage	Target latency
camera enumerate/probe	< 2 s on startup
frame gate decision	5-20 ms/frame
OCR on candidate frame	40-150 ms
VLM inference on candidate frame	150-800 ms depending profile
partial STT transcript	< 500 ms
short utterance final STT	< 1.8 s
TTS first audible output	< 700 ms

Stage	Target latency
memory lookup top-k	20-120 ms
tool dispatch overhead	< 100 ms before tool runtime

10.10 Adaptive runtime policy

The perception stack should not analyze every frame fully. Use these defaults:

- idle mode: 2 FPS capture
- watchful mode: 5 FPS capture
- active burst mode: 10 FPS capture

Escalation triggers:

- user is actively in a session
- strong scene change detected
- explicit capture or command trigger

De-escalation triggers:

- no interaction
- thermal pressure
- low battery or low power profile

10.11 Power and resource planning

These values are engineering planning targets only and should be validated on real hardware:

State	Planning target
idle monitoring	keep average low enough for always-on desk mode
active camera + inference	bursty, not continuous full-frame inference
voice roundtrip	prioritized over background summarization
degraded mode	preserve memory, UI, and typed workflows even if camera/audio budget is reduced

Resource priorities:

1. maintain app control and service health
2. preserve memory writes and retrieval
3. preserve user-initiated voice and typed interactions
4. reduce background perception first under pressure

10.12 Full software stack table

Layer	Technology	Purpose
desktop shell	Tauri v2	user-facing app
UI	React + TypeScript	views, onboarding, health, logs
native host layer	Rust	app-side native integration
orchestration	OpenClaw Gateway	sessions and tools
camera/media	V4L2 + GStreamer + OpenCV	capture and preprocessing
perception models	LFM2.5-VL-450M + Gemma fallback	scene understanding
STT	whisper.cpp	offline transcription
TTS	KittenTTS	offline speech output
memory store	SQLite	canonical structured memory
memory mirror	Markdown/text	inspectable memory export
semantic retrieval	local vector index	context lookup
guarded execution	os-toold	audited OS access
packaging	Debian package tooling	signed `.deb` distribution
service manager	systemd	service lifecycle

11. Repo Shape and Engineering Pipeline

11.1 Target repo shape

```
apps/
  dan-glasses-app/
shared/
  contracts/
  schemas/
services/
  openclaw-gateway/
  perceptiond/
  audiod/
  memoryd/
  os-toold/
tools/
  dan-glassesctl/
packaging/
  debian/
docs/
  architecture/
  manifests/
```

11.2 Ownership by subsystem

This is the intended implementation split:

- `apps/dan-glasses-app`
  - installer UX
  - bootstrap wizard
  - logs/health UI
  - settings and overrides
- `shared/contracts`
  - IPC types
  - health schemas
  - bootstrap schema
  - policy schema
- `services/openclaw-gateway`
  - session orchestration
  - tool routing
  - action policy integration
- `services/perceptiond`
  - camera enumeration
  - frame capture
  - OCR and VLM dispatch
- `services/audiod`
  - device enumeration
  - transcription
  - speech output
- `services/memoryd`
  - schema initialization
  - memory writes
  - retrieval APIs
- `services/os-toold`
  - command execution
  - policy enforcement
  - audit records
- `packaging/debian`
  - control files
  - maintainer scripts
  - desktop entry
  - systemd units

11.3 Build matrix

- `x86_64-unknown-linux-gnu`
- `aarch64-unknown-linux-gnu`

11.4 Engineering pipeline

1. build Tauri app
2. build sidecar/service binaries
3. assemble install tree
4. inject Debian packaging metadata/scripts
5. build `.deb`
6. sign package
7. publish package and metadata
8. publish model manifest

11.5 CI expectations

- unit tests for service logic
- hardware probe tests where possible
- packaging smoke tests
- install/upgrade/purge tests
- offline startup tests

11.6 Developer implementation workflow

flowchart LR A[Update contracts in shared/contracts] --> B[Implement or modify service] B --> C[Run unit tests] C --> D[Run local integration checks] D --> E[Build Tauri app and sidecars] E --> F[Assemble Debian package] F --> G[Run install/upgrade/offline smoke tests] G --> H[Publish signed package and manifests]

12. Release Pipeline

12.1 Packaging pipeline

1. build Tauri supervisor UI
2. bundle service binaries into `/opt/dan-glasses/`
3. generate `.deb`
4. validate maintainer scripts
5. sign package
6. publish to APT repository

12.2 Release artifacts

Each release should publish:

- signed `.deb` for `x86_64`
- signed `.deb` for `aarch64`
- `Packages` and `Release` metadata for the APT repo
- detached checksums/signatures where applicable
- model manifest files

12.3 Update pipeline

- update delivered through APT repository
- app can notify user that a package upgrade is available
- package upgrade is the system of record for runtime updates

12.4 Upgrade requirements

- preserve `/etc/dan-glasses/`
- preserve `/var/lib/dan-glasses/`

- preserve downloaded models where compatible
- re-run `postinst` safely
- restart affected services gracefully

12.5 Release and update flow

flowchart TD A[Commit merged] --> B[CI build x86_64 and aarch64 artifacts] B --> C[Bundle Tauri app + sidecars] C --> D[Generate signed .deb packages] D --> E[Publish APT repo metadata] E --> F[User apt update && apt upgrade] F --> G[Package upgrade installs] G --> H[Maintainer scripts run safely] H --> I[Services restart] I --> J[App shows new version and health]

13. Implementation Phases

Phase 0: Packaging and bootstrap skeleton

- create Tauri app shell
- create package layout
- define service units
- create `postinst`, `prerm`, `postrm`
- implement bootstrap wizard UI

Phase 1: Device discovery and health

- implement V4L2 enumeration
- implement audio device discovery
- implement service health reporting
- implement `dan-glassesctl`

Phase 2: Voice and memory core

- integrate `whisper.cpp`
- integrate `KittenTTS`
- implement SQLite schema
- implement Markdown mirror
- implement vector index bootstrap

Phase 3: Perception and model runtime

- implement frame pipeline
- implement OCR branch
- integrate `LFM2.5-VL-450M`
- implement Gemma fallback
- emit `PerceptionObservation` events

Phase 4: OpenClaw tools and policy

- wire memory, voice, camera, and OS tools
- implement command policy profiles
- implement audit log and kill switch

Phase 5: Release readiness

- package signing
- APT repository publishing
- install/upgrade/purge automation
- laptop and Redax validation

14. Acceptance Criteria and Test Plan

14.1 Install and bootstrap

- install on Redax Debian/Ubuntu with `sudo` creates all required paths and services
- install on Linux laptop works with integrated webcam and USB webcam
- first-run bootstrap downloads models, verifies checksums, and resumes interrupted downloads

14.2 Runtime health

- OpenClaw starts and receives perception and voice events
- service health endpoints correctly report healthy/degraded states
- offline startup works after initial model install

14.3 Camera and audio

- V4L2 enumeration correctly lists multiple cameras
- default camera selection is sane and user-overridable
- `whisper.cpp` and `KittenTTS` work fully offline

14.4 Memory

- SQLite and Markdown mirror remain consistent across restart/crash recovery
- memory lookup works locally without network access
- optional Obsidian/Mem0/MemPalace adapters do not break canonical memory writes

14.5 Developer tooling

- risky OS commands are blocked or confirmed based on permission profile
- `git status`, file edits, and process restart flows are auditable
- emergency kill switch halts running command tasks

14.6 Upgrade and recovery

- package upgrade via APT preserves config and memory
- idempotent install hooks succeed on reinstall
- degraded mode activates correctly when hardware or models are missing

14.7 Additional acceptance checks

- the app can show every service status without opening a terminal
- the app can explain why it is degraded and how to recover
- a developer agent can read this file alone and reconstruct:
  - product intent
  - install strategy
  - service topology
  - memory design
  - tooling model
  - release and test strategy

15. Risks, Deferred Decisions, and Non-Goals

15.1 Known risks

- local model performance may vary sharply across Redax and laptop hardware
- camera auto-classification heuristics may need per-vendor tuning
- TTS/STT quality may require profile tuning across devices

15.2 Deferred decisions

- dedicated custom Linux image
- wearable-only UX polish

- advanced wake-word pipeline
- promoting Obsidian, Mem0, MemPalace, or Hermes integrations to first-class memory backend

15.3 Explicit non-goals for v1

- cloud-first orchestration
- Flatpak/AppImage as the primary shipping path
- mandatory custom Redax image
- per-camera bespoke code instead of a generic V4L2 provider layer

16. Consolidation Status

This file is now the only canonical markdown document for Dan Glasses product, research, system design, developer handoff, and OpenClaw prompt guidance.

The previously separate research notes and OpenClaw prompt content have been merged into this file. There should be no parallel architecture authority outside this document.

17. Research Grounding and Consolidated Findings

17.1 Research grounding matrix

Category	Finding	Status	Impact on v1
DanLab product direction	DanLab publicly positions AI glasses with multimodal overlays	sourced	Confirms product category, not implementation details
OpenClaw architecture	OpenClaw is a self-hosted gateway/node runtime over WebSocket	sourced	Supports OpenClaw-first local orchestration
Hermes provider model	Hermes allows one external memory provider at a time while built-in memory remains active	sourced	Limits direct Hermes-centered multi-provider design
Hermes mem0 docs	Hermes mem0 provider docs are API-key/cloud-oriented	sourced	Prevents mem0-through-Hermes as strict offline canonical core
Mem0 itself	Mem0 can still be run locally outside Hermes default provider assumptions	sourced/inferred	Keeps Mem0 viable as optional adapter
Obsidian MCP	Excellent human-readable workflow and note surface	inferred from tool shape and use pattern	Best as mirror/workflow layer, not canonical high-frequency store
MemPalace	Interesting advanced long-term memory candidate	sourced/project-level	Keep as optional research adapter for later validation
Canonical v1 memory	SQLite + Markdown mirror + local vector index is the strongest offline core	engineering decision	Locked for v1
Install strategy	`.deb` + `systemd` is more viable than Flatpak/AppImage for this product	engineering decision grounded in Linux packaging constraints	Locked for v1
Runtime shell	Tauri supervisor app is the best balance of native packaging, UI, and sidecar orchestration	engineering decision	Locked for v1

17.2 Consolidated research conclusions

- The product should be treated as a **single Linux app experience** backed by a managed local service mesh.
- OpenClaw remains the only orchestration runtime in v1; Hermes is not the canonical runtime path.
- The app must support both Redax hardware and normal Linux laptops using one architecture with two package builds.
- The safest v1 path is to make laptop and desk workflows first-class, while preserving the same service contracts for Redax deployment.
- Camera support should remain generic through V4L2 with adapters, not vendor-specific bespoke paths.
- Model bootstrap belongs in first run, not inside the `.deb`.

17.3 Local data ownership and privacy policy

- All core memory persists locally first.
- Sync and export are optional and explicit.

- Raw media retention is off by default.
- SQLite is the authoritative store for structured data.
- Markdown is the authoritative human-readable mirror.
- Optional Obsidian export is a mirror only, not the source of truth.
- If privacy strict mode is enabled, only summaries and structured memory rows are retained.

17.4 Failure handling matrix

Failure	Required behavior
No camera found	Enter `camera_degraded` mode and preserve voice/text workflows
No microphone found	Enter `audio_degraded` mode and preserve typed workflows
Model download interrupted	Resume using manifest and bootstrap state files
LFM unavailable	Route requests to Gemma fallback profile
Memory write failure	Spool to durable journal and replay
`os-toold` policy failure	Fail closed for mutating operations
Service restart/crash	Recover under `systemd` and show degraded reason in UI
Low disk	Offer reduced model profile and block unsafe writes
Offline boot after initial setup	Continue with installed models and local memory only

17.5 Memory provider comparison and adoption rules

Provider	v1 role	Strengths	Weaknesses	Adoption phase
SQLite + Markdown + vectors	canonical core	offline, inspectable, durable, low complexity	requires custom adapter work	v1 required
Obsidian MCP	optional mirror/workflow layer	human-readable, graph-oriented, good for developer notes	weak as sole event store	v1 optional
Mem0	optional experimental adapter	promising retrieval and memory abstraction, can run locally	Hermes integration path is cloud-oriented by default	v1 research / v1.5 candidate
MemPalace	optional advanced research adapter	long-term memory framing and retrieval concepts	added complexity, needs validation	v1 research / v1.5 candidate
Hermes provider integration	optional compatibility layer	useful if a later skill/memory workflow demands it	one-provider-at-a-time constraint complicates canonical core	defer unless needed

Promotion rule for any optional provider:

- must beat canonical core on p95 read/write latency
- must improve retrieval quality for developer workflows
- must survive reboot and crash recovery cleanly
- must not replace inspectability or offline reliability

17.6 Additional engineering guidance pulled from the merged drafts

- `perceptiond` should prefer adaptive capture: 2 FPS idle, 5 FPS watchful, 10 FPS active burst.
- OCR should be a first-class branch for terminal/editor/log-heavy scenes.
- `audiod` should support partial transcripts to reduce latency before OpenClaw reasoning begins.
- Voice defaults should be push-to-talk first; wake behavior remains experimental.
- TTS should always have a UI fallback so a speech issue does not block the workflow.

18. OpenClaw Master Prompt Appendix

Use the following appendix prompt if another agent or OpenClaw instance needs to regenerate or refine the full Dan Glasses architecture without re-reading chat history.

```
# OpenClaw Master Prompt: Dan Glasses Developer-First Linux App Architecture

You are the lead systems architect for Dan Glasses by DanLab.

Design a developer-first, offline-first, single Linux app architecture where OpenClaw is the only orchestration runtime.

## Mission

Produce an implementation-ready architecture for Dan Glasses with:

- one Linux desktop app as launcher/supervisor
- OpenClaw runtime on-device
- camera/webcam perception pipeline
- voice pipeline using `whisper.cpp` + `KittenTTS`
- local memory-first storage and retrieval
- guardrailed broad Linux OS tool access for developer workflows

## Non-negotiable constraints

- OpenClaw orchestration only
- STT must use `whisper.cpp`
- TTS must use `KittenTTS`
- Vision model: `LFM2.5-VL-450M` primary, Gemma fallback
- Camera ingest via V4L2/GStreamer on Redax-class Linux board
- Core system must work offline after bootstrap
- Default memory ownership is local-first
- Raw media retention must be opt-in
- Shipping shell is `Tauri`
- Install strategy is signed `.deb` + `systemd`

## Required services

- `dan-glasses-app`
- `openclaw-gateway`
- `perceptiond`
- `audiod`
- `memoryd`
- `os-toold`
- optional `dan-glassesctl`

## Developer tooling requirements

Define OpenClaw tool contracts for:

- `os.exec`, `os.read`, `os.write`
- `git.*`
- `proc.*`
- `net.*`
- `memory.*`
- `hud.*`
- `voice.*`

Define permission profiles:

- `readonly`
- `dev-standard`
- `admin-guarded`

Include:
```

```
- command policy and guardrails
- audit logging
- emergency kill switch

## Memory requirements

- v1 canonical memory: SQLite + local Markdown/text mirror + local vector index
- Obsidian MCP, Mem0, and MemPalace are optional adapters, not mandatory core
- explain why Hermes external provider constraints prevent it from being the canonical multi-provider memory core
- define v1.5 promotion gates for optional memory providers

## Audio and perception requirements

Include:

- `stt.transcribe(audio_chunk)`
- `tts.speak(text, voice, rate)`
- `PerceptionObservation` containing:
  - `source = camera.webcam`
  - `dev_context`
  - `ocr_text`
  - `action_hint`

## Deliverables

Produce:

1. Executive summary
2. Research grounding matrix
3. Linux app process map
4. Startup/control flow
5. Camera-perception flow
6. Voice roundtrip flow
7. Memory write/read flow
8. IPC contract table
9. Tool contract and permission matrix
10. Memory provider comparison matrix
11. Local storage and retention policy
12. Failure and fallback strategy
13. Test scenarios and acceptance criteria

## Output rules

- Use Markdown
- Use Mermaid for diagrams
- Separate sourced facts, engineering decisions, and deferred choices
- Stay implementation-oriented
- Do not redesign into a cloud-first app
```

19. References

Product and orchestration

1. DanLab: https://www.danlab.dev/ (https://www.danlab.dev/)
2. OpenClaw docs: https://docs.openclaw.ai/ (https://docs.openclaw.ai/)
3. OpenClaw gateway protocol: https://docs.openclaw.ai/gateway/protocol (https://docs.openclaw.ai/gateway/protocol)
4. OpenClaw nodes: https://docs.openclaw.ai/nodes (https://docs.openclaw.ai/nodes)

Linux packaging and services

1. Debian maintainer scripts policy: https://www.debian.org/doc/debian-policy/ch-maintainerscripts.html (https://www.debian.org/doc/debian-policy/ch-maintainerscripts.html)
2. Debian maint-guide: https://www.debian.org/doc/manuals/maint-guide/ (https://www.debian.org/doc/manuals/maint-guide/)
3. systemd service reference: https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html (https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html)
4. systemd unit reference: https://www.freedesktop.org/software/systemd/man/latest/systemd.unit.html (https://www.freedesktop.org/software/systemd/man/latest/systemd.unit.html)

App shell and UI

1. Tauri docs: https://v2.tauri.app/ (https://v2.tauri.app/)

Camera and media

1. Linux V4L2 userspace API: https://docs.kernel.org/userspace-api/media/v4l/v4l2.html (https://docs.kernel.org/userspace-api/media/v4l/v4l2.html)
2. Linux media controller API: https://docs.kernel.org/userspace-api/media/mediactl/media-controller.html (https://docs.kernel.org/userspace-api/media/mediactl/media-controller.html)
3. GStreamer docs: https://gstreamer.freedesktop.org/documentation/ (https://gstreamer.freedesktop.org/documentation/)
4. OpenCV docs: https://docs.opencv.org/ (https://docs.opencv.org/)

Models and AI runtime

1. LFM2.5-VL-450M model card: https://huggingface.co/LiquidAI/LFM2.5-VL-450M (https://huggingface.co/LiquidAI/LFM2.5-VL-450M)
2. llama.cpp: https://github.com/ggml-org/llama.cpp (https://github.com/ggml-org/llama.cpp)
3. whisper.cpp: https://github.com/ggml-org/whisper.cpp (https://github.com/ggml-org/whisper.cpp)
4. KittenTTS: https://github.com/soldier444xd/KittenTTS (https://github.com/soldier444xd/KittenTTS)

Memory and adapters

1. Mem0: https://github.com/mem0ai/mem0 (https://github.com/mem0ai/mem0)
2. Mem0 local companion example: https://docs.mem0.ai/cookbooks/companions/local-companion-ollama (https://docs.mem0.ai/cookbooks/companions/local-companion-ollama)
3. FAISS: https://github.com/facebookresearch/faiss (https://github.com/facebookresearch/faiss)
4. Obsidian MCP server index: https://github.com/modelcontextprotocol/servers (https://github.com/modelcontextprotocol/servers)
5. MemPalace: https://github.com/MemPalace/mempalace (https://github.com/MemPalace/mempalace)
6. Hermes memory providers: https://hermes-agent.nousresearch.com/docs/user-guide/features/memory-providers (https://hermes-agent.nousresearch.com/docs/user-guide/features/memory-providers)

Protocols

1. Anthropic MCP overview: https://www.anthropic.com/news/model-context-protocol (https://www.anthropic.com/news/model-context-protocol)
2. Anthropic MCP docs: https://docs.anthropic.com/en/docs/mcp (https://docs.anthropic.com/en/docs/mcp)
3. MCP specification: https://modelcontextprotocol.io/specification (https://modelcontextprotocol.io/specification)